

```

# Update the plan based on feedback
def update_plan(user_id: int, updated_text: str):
    db = SessionLocal()
    workout = db.query(WorkoutPlan).filter_by(user_id=user_id).first()
    if workout:
        workout.updated_plan = updated_text
        db.commit()
    db.close()

# Fetch the original plan for a user
def get_original_plan(user_id: int):
    db = SessionLocal()
    plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user_id).first()
    return plan.original_plan if plan else None

def get_user(user_id: int):
    db = SessionLocal()
    return db.query(User).filter(User.id == user_id).first()

```

## Admin View of All Users

Route: `/view-all-users`

Template: `all_users.html`

Displays a list of all users and their respective plans. Rendered with Jinja2 using a `{% for user in users %}` loop, it allows administrators to see original and updated plans clearly in a structured table format.

```

# 8. Web: View all users & their plans
@router.get("/view-all-users", response_class=HTMLResponse)
def view_all_users(request: Request):
    db = SessionLocal()
    users = db.query(User).all()
    user_data = []
    for user in users:
        plan = db.query(WorkoutPlan).filter(WorkoutPlan.user_id == user.id).first()
        user_data.append({
            "id": user.id,
            "name": user.name,
            "age": user.age,
            "weight": user.weight,
            "goal": user.goal,
            "intensity": user.intensity,
            "original_plan": plan.original_plan if plan else "N/A",
            "updated_plan": plan.updated_plan if plan and plan.updated_plan else "Not updated"
        })
    db.close()
    return templates.TemplateResponse("all_users.html", {
        "request": request,
        "users": user_data
    })

```

## Activity 2.2: Implement the FastAPI Backend to Manage Routing and User Input Processing

### Define Routes in FastAPI

All routing logic is handled in `routes.py`. Each route is linked to the corresponding function developed in Activity 2.1.

### Process User Input

- HTML Form: Created in `index.html`
- Users enter: `username`, `user_id`, `age`, `weight`, `goal`, and `intensity`
- FastAPI captures inputs using `Form(...)` parameters
- Pydantic schemas (`UserInput`, `FeedbackRequest`) are used to validate structure

### Integrate Gemini API Calls

Each route handler connects directly to the respective Gemini-powered function:

- `/generate-workout` –
  - `generate_workout_gemini()` for workout plan
  - `generate_nutrition_tip_with_flash()` for tip
- `/submit-feedback` –
  - Retrieves plan
  - Sends original + feedback to `update_workout_plan()`

All AI-generated responses are returned cleanly and injected into the frontend templates for display.

## Milestone 3. `routes.py` Development

Milestone 3 focuses on building the routing logic inside the `routes.py` file, which acts as the core operational layer of the FitBuddy application. It bridges the frontend templates, AI generation modules (Gemini Pro and Flash), and the SQLite database. Each route defines how user inputs are collected, processed, and responded to — creating a dynamic, personalized experience for fitness planning and feedback-based updates.

## Activity 3.1: Writing the Main Application Logic in `routes.py`

### Defining Core routes in `routes.py`:

Separate routes were implemented to serve each of FitBuddy's main functionalities:

1. `/` : Displays the user form via `index.html`
2. `/generate-workout`: Processes user input and generates a personalized workout plan
3. `/submit-feedback`: Updates the workout plan based on user feedback
4. `/view-all-users`: Displays an admin dashboard showing all users and their plans

### Set up Route Handlers for Each Feature

➤ `/` – Home Route

- Returns the homepage (`index.html`)
- No form processing logic

#### ➤ `/generate-workout` – Plan Generator

- Receives user input using FastAPI's `Form(...)`
- Constructs a `UserInput` Pydantic model
- Calls:
  - `generate_workout_gemini(...)` → Generates workout plan via Gemini 1.5 Pro
  - `generate_nutrition_tip_with_flash(...)` → Gets nutrition tip via Gemini Flash
- Stores:
  - User details with `save_user(...)`
  - Plan with `save_plan(...)`
- Returns `result.html` with dynamic data

#### ➤ `/submit-feedback` – Update Plan with Feedback

- Captures feedback and `user_id`
- Retrieves original plan from DB
- Uses `update_workout_plan(...)` to revise the plan via Gemini 1.5 Pro
- Updates the revised plan in the DB using `update_plan(...)`
- Renders updated result on `result.html`

#### ➤ `/view-all-users` – Admin View

- Calls `get_all_users()` and `get_all_plans()` to fetch DB records
- Passes them to `all_users.html`
- Admin can view:
  - Username, Age, Weight, Goal, Intensity
  - Original and updated workout plans

### Integrate Gemini AI Responses in Each Function

Each AI-related route calls Google's Gemini models with specific prompts:

- Workout Plan Generation:  
`generate_workout_gemini()` sends fitness goal and intensity to Gemini Pro to return a structured 7-day workout plan.
- Nutrition Tip Generation:  
`generate_nutrition_tip_with_flash()` uses Gemini Flash to generate a practical tip related to the user's fitness goal.
- Workout Plan Update via Feedback:  
`update_workout_plan()` passes original plan + user feedback to Gemini Pro and returns a revised plan.

All responses are returned to the user through `result.html`, maintaining clarity using `<pre>` blocks for formatting.

### Workout generation route Nutrition/Recovery tip generation route